ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

Стек протоколов. Транспортный уровень



Разработка
Сетевые технологии

03

Сетевые технологии · лекция 3

Транспортный уровень

Как данные доходят до другого процесса
и почему передача может остановиться

Наше место в модели OSI

После смысла и представления сообщений разбираем механизмы доставки.

Уровень	Задача	В курсе
L7	Прикладной	Лекция 1: HTTP, DNS, почта
L6	Представления	Лекция 2: форматы, сериализация, защита
L5	Сеансовый	Сегодня: отдельно обозначим задачи диалога
L4	Транспортный	Сегодня: доставка между конечными точками
L3	Сетевой	Доставка IP-пакетов, далее по курсу
L2	Канальный	Передача в канальном соединении
L1	Физический	Передача сигналов

> OSI разделяет функции. Реальные протоколы могут выполнять задачи нескольких уровней.

От содержимого сообщения к его доставке

В прошлый раз мы разбирали тело HTTP и письмо. Но как получатель получил их байты?

Уже умеем объяснять	Теперь нужно понять
Content-Length и chunked	Почему одно чтение возвращает лишь часть сообщения?
TLS и проверку сертификата	Что происходит до TLS и при обрыве соединения?
Фреймы и потоки HTTP/2	Почему потеря задерживает несколько обменов?

> Привычный HTTPS: HTTP → TLS → TCP → IP. Сегодня раскрываем TCP и альтернативы.

Разным приложениям нужны разные свойства

Задача	Что важно получателю
Файл или письмо	Получить всё содержимое без пропусков
DNS-запрос	Сопоставить ответ с вопросом и обработать отсутствие ответа
Разговор по сети	Получать звук вовремя, учитывать потери и задержку

> После занятия: различать свойства транспорта, читать обмен и объяснять причины ожидания.

Выбор протокола зависит от всей системы. «Голос всегда UDP» и «DNS только UDP» не работают как правила.

Адрес узла и порт приложения

На одном компьютере работают несколько сетевых сервисов. IP-адреса для выбора получателя недостаточно.

```
Клиент                                  Сервер
192.0.2.10:53000 ── TCP ──▶ 198.51.100.20:443
```

IP-адрес	Порт	Сокет
Адрес узла в IP-сети	Часть транспортной конечной точки	Интерфейс приложения к сетевому стеку

> Порт подсказывает ожидаемую службу, но не доказывает, какой протокол она использует.

Один серверный порт, много соединений

Клиент	Сервер	Протокол
`192.0.2.10:53000`	`198.51.100.20:443`	TCP
`192.0.2.11:53000`	`198.51.100.20:443`	TCP
`192.0.2.10:53001`	`198.51.100.20:443`	TCP

Соединение различают по адресам и портам обеих сторон. Для сетевого потока учитывают также транспортный протокол.

> TCP/53 и UDP/53 относятся к разным транспортам. За NAT адреса могут зависеть от точки наблюдения.

Какие транспортные протоколы разберём

Все они доставляют данные между приложениями, но предоставляют разные свойства.

Протокол	Главная идея
TCP	Надёжный упорядоченный поток байтов
UDP	Отдельные датаграммы без встроенного восстановления потерь
SCTP	Надёжные сообщения, несколько потоков и адресов
DCCP	Датаграммы с управлением перегрузкой без восстановления потерь

> Начнём с UDP, затем подробно разберём TCP и сравним его с SCTP.

На примере DCCP обсудим трудности внедрения нового транспорта.

UDP переносит отдельные датаграммы

Знакомый пример: классический DNS-запрос и отдельная датаграмма с ответом.

```
Клиент:53000 ── UDP / DNS query ──▶ resolver:53
Клиент:53000 ◀── UDP / DNS answer ── resolver:53
```

Порты	Length	Checksum
Конечные точки обмена	Длина UDP-заголовка и данных	Контроль ошибок в покрываемых данных

UDP — User Datagram Protocol. Собственного handshake перед каждой датаграммой нет.

Каких обещаний UDP не даёт

Событие	Поведение UDP
Пакет потерялся	Нет встроенного восстановления
Пакеты поменялись местами	Нет встроенной выдачи в исходном порядке
Сеть перегружена	Нет встроенного управления нагрузкой
Данные изменились	Есть checksum, но нет криптографической защиты

> Подтверждения, повторы и управление скоростью может реализовать протокол поверх UDP.

Для голоса важна своевременность

Фрагмент звука, пришедший после момента воспроизведения, может уже не понадобиться.

Характеристика	Что слышит пользователь
Большая задержка	Собеседники начинают перебивать друг друга
Jitter: меняющееся время доставки	Неровное поступление данных требует буферизации
Потери	Пропуски, которые медиасистема пытается скрыть или восстановить

> UDP проходит через те же очереди. Отсутствие handshake не гарантирует низкую задержку.

Большая датаграмма может столкнуться с фрагментацией IP. Размеры тоже имеют значение.

TCP предоставляет поток байтов

TCP — Transmission Control Protocol. Приложение получает данные в порядке отправки, а не в порядке прихода сегментов.

Предоставляет	Не обещает
Упорядоченный поток в каждом направлении	Границы сообщений и вызовов send
Восстановление пропусков, пока обмен возможен	Доставку к заданному сроку при любых отказах
Два направления передачи	Выполнение сервером прикладной операции

> При неисправимом нарушении соединение может завершиться ошибкой.

Две записи могут дать три чтения

Отправитель передаёт `PING\n` и `PONG\n`. Разделитель сообщений — LF.

Вариант	Результаты чтений у получателя
A	`"PING\n"`, `"PONG\n"`
B	`"PI"`, `"NG\nPO"`, `"NG\n"`
C	`"PING\nPONG\n"`

В каких вариантах получатель восстановит оба сообщения? Что ему нужно сохранить между чтениями?

> Все варианты допустимы. Границы сообщений определяет прикладной протокол, а не размер прочитанного блока.

Поток передаётся сегментами

```
Сообщения приложения
  TCP: последовательность байтов → сегменты
    IP: пакеты → канальное соединение
```

MTU	MSS
Maximum Transmission Unit	Maximum Segment Size
Ограничение размера пакета на участке	Размер данных TCP без заголовков TCP/IP

> При MTU 1500, IPv4 20 байт и TCP 20 байт для данных остаётся 1460. Эти предпосылки могут меняться.

Сегментация TCP и фрагментация IP — разные механизмы.

Структура заголовка TCP

Базовый формат TCP: первые пять строк — по 32 бита. Размеры полей указаны в битах.

<table>
<tr><td colspan="2">Порт отправителя (16)</td><td colspan="2">Порт получателя (16)</td></tr>
<tr><td colspan="4">Номер последовательности — SEQ (32)</td></tr>
<tr><td colspan="4">Номер подтверждения — ACK (32)</td></tr>
<tr><td>Длина
загол. (4)</td><td>Резерв
(4)</td><td>Флаги (8)
CWR ECE URG ACK
PSH RST SYN FIN</td><td>Окно получателя — Window (16)</td></tr>
<tr><td colspan="2">Контрольная сумма (16)</td><td>Указатель срочных данных (16)</td></tr>
<tr><td colspan="4">Опции и выравнивание: от 0 до 40 байт · MSS, SACK, Window Scale…</td></tr>
</table>

> Заголовок занимает 20–60 байт. После него идут данные TCP-сегмента.

ACK-флаг и номер ACK — разные поля. ISN передаётся в SEQ при установлении; cwnd в заголовке нет.

Номера относятся к байтам

Учебный обмен: клиент отправил серверу 500 байт.

```
Клиент → сервер: SEQ=1001 LEN=500
                 байты 1001 ... 1500
Сервер → клиент: ACK=1501
```

SEQ	ACK — Acknowledgment
Номер первого байта данных сегмента	Номер следующего ожидаемого байта противоположного направления

> У двух направлений свои последовательности. ACK=1501 подтверждает непрерывный префикс до 1500.

Установление TCP-соединения

Стороны начинают с ISN — Initial Sequence Number. SYN означает Synchronize.

Клиент		Сервер
	`SYN · SEQ=1000 →`	
	`← SYN, ACK · SEQ=7000 ACK=1001`	
	`ACK · SEQ=1001 ACK=7001 →`	

SYN занимает одну позицию в пространстве номеров. У сторон отдельные начальные номера.

> TCP handshake устанавливает транспортное соединение. В обычном HTTPS затем начинается TLS handshake.

Ранние генераторы ISN были предсказуемыми. Позже добавили псевдослучайную составляющую, чтобы затруднить угадывание номеров и подделку сегментов.

Три результата попытки соединения

Точка наблюдения: захват у клиента. Последовательности упрощены.

Видимый обмен	Что известно	Что ещё не доказано
SYN, SYN+ACK, ACK	Наблюдаем установление TCP	Готовность приложения
SYN, затем RST	Получили отказ/сброс	Какой узел и почему его сформировал
SYN, SYN, SYN…	Не видим ответа	Где исчез пакет или обратный ответ

> Один захват сужает поиск, но не показывает автоматически всю сеть.

Подтверждение байтов и результат операции

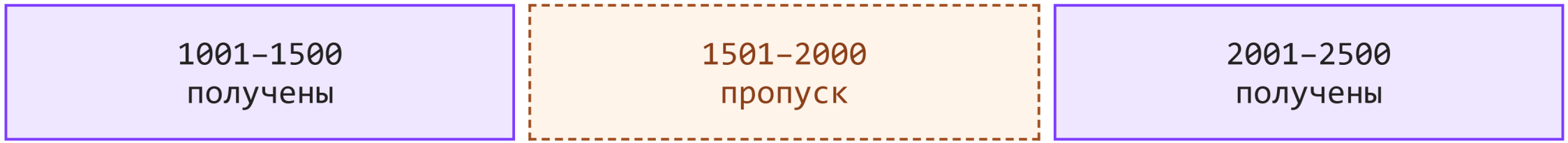
Наблюдение	Что оно подтверждает
Приложение передало байты сетевому стеку ОС	ОС приняла эти байты для отправки; доставка получателю ещё не подтверждена
TCP ACK покрыл запрос	Принимающий TCP подтвердил данные
Получен ответ приложения	Есть результат по правилам прикладного протокола

> После TCP ACK ещё неизвестно, прочитал ли запрос обработчик и выполнил ли операцию.

Этот вопрос вернётся в блоке RPC: что делать, если ответ не пришёл?

Пакеты могут прийти с пропуском

Сервер получил два диапазона байтов. Средний ещё не пришёл.



Возможны потеря, задержка или перестановка пакетов. TCP должен восстановить непрерывную последовательность.

> Получение более поздних байтов не заполняет пропуск автоматически.

ACK и SACK: что узнал отправитель

Сервер получил 1001–1500 и 2001–2500. Между ними пропуск 1501–2000.

Только ACK	ACK + SACK
`ACK = 1501`	`ACK = 1501` `SACK = [2001, 2501)`
«Всё до 1500 получено. Следующий ожидаемый байт 1501»	«Жду 1501. Дополнительно сообщаю: байты 2001–2500 уже получены»
Из значения ACK не видно, что пришло за пропуском	Отправитель точнее выбирает диапазоны для повторной передачи

После прихода 1501–2000 непрерывный диапазон вырастет: `ACK = 2501`.

> SACK помогает повторить недостающее. Приложение ждёт заполнения пропуска.

SACK (Selective Acknowledgment) — выборочное подтверждение, опция TCP.
Поддержку объявляют в SYN. Правая граница SACK не включается.

Как отправитель решает повторить данные

Механизм	Смысл
RTT — Round-Trip Time	Время пути туда и обратно
RTO — Retransmission Timeout	Ожидание перед повторной передачей, учитывающее RTT и его вариацию
Fast retransmit	Повтор по дополнительным сигналам, не дожидаясь RTO

> Три duplicate ACK — классический учебный пример. Реальные реализации используют и другие механизмы.

Повторная передача сама по себе не доказывает перегрузку сети.

Какие байты получит приложение?

Разбор: входящий TCP-поток сервера. Диапазон после пропуска уже сохранён.



До заполнения пропуска	После заполнения
Доступен префикс до 1500	Новый непрерывный участок до 2500

> Head-of-Line blocking (HOL): последующие байты ждут недостающую часть упорядоченного потока.

Пакеты, ACK и обратное направление могут продолжать движение.

Потоки HTTP/2 разделяют один TCP-поток

В лекции 2 сервер чередовал фреймы двух HTTP/2-потоков. На уровне TCP это одна последовательность байтов.



Фрейм B может ждать выдачи, хотя относится к другому прикладному обмену.

> Разные HTTP/2 streams не устраняют ожидание пропуска внутри общего TCP-потока.

Это другой механизм, чем порядок ответов при HTTP/1.1 pipelining.

FIN завершает одно направление

Клиент закончил отправку, но ещё может получать оставшийся ответ.

Клиент		Сервер
	`FIN →`	
	`← ACK`	
	`← данные ответа, затем FIN`	
	`ACK →`	

> Half-close: одно направление уже закрыто, другое ещё работает. Число пакетов завершения может различаться.

FIN и RST показывают разные события

FIN	RST — Reset
Сторона закончила отправку	Соединение сброшено
Предшествующие данные идут по порядку	Продолжение обмена может завершиться ошибкой
Возможен ответ в другом направлении	Для причины нужны дополнительные наблюдения

> Сброс мог сформировать конечный узел или промежуточное устройство. Один RST не назначает виновника.

После закрытия может оставаться состояние

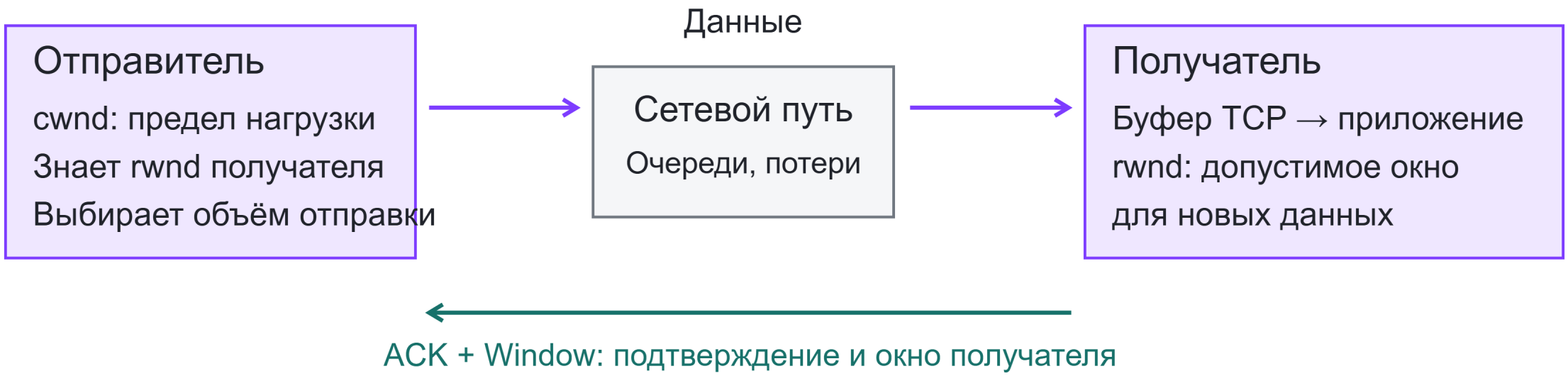
Состояние	Что означает	На что обратить внимание
TIME_WAIT	Учёт поздних сегментов и завершающего обмена	Нормальное состояние, оцениваем объём и ресурсы
CLOSE_WAIT	FIN получен, местная сторона ещё не завершила отправку	Долгое накопление требует проверки приложения

> Не всякое оставшееся соединение является утечкой.

TCP keepalive, прикладной heartbeat и HTTP keep-alive решают разные задачи. Сравнение есть в лонгриде.

Два окна ограничивают одну отправку

Получателю нужно место для данных, а сети — посильная нагрузка. Отправитель учитывает оба ограничения.



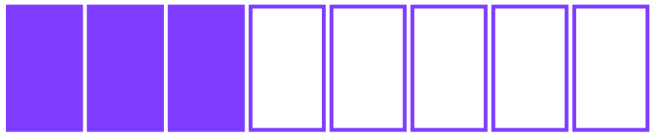
> rwnd защищает буфер получателя. cwnd помогает отправителю регулировать нагрузку на путь.

«В полёте» — байты, отправленные, но ещё не подтверждённые TCP. Они могут уже быть у получателя, пока ACK идёт обратно.

rwnd: сколько готов принять получатель

Receive window — окно получателя. Учебный буфер вмещает 8 одинаковых блоков.

1. Есть место



rwnd = 5 блоков

Принимаем новые данные

2. Приложение не читает



rwnd = 0 блоков

Новые данные ждут

3. Приложение прочитало



rwnd = 4 блоков

Можно открыть окно снова

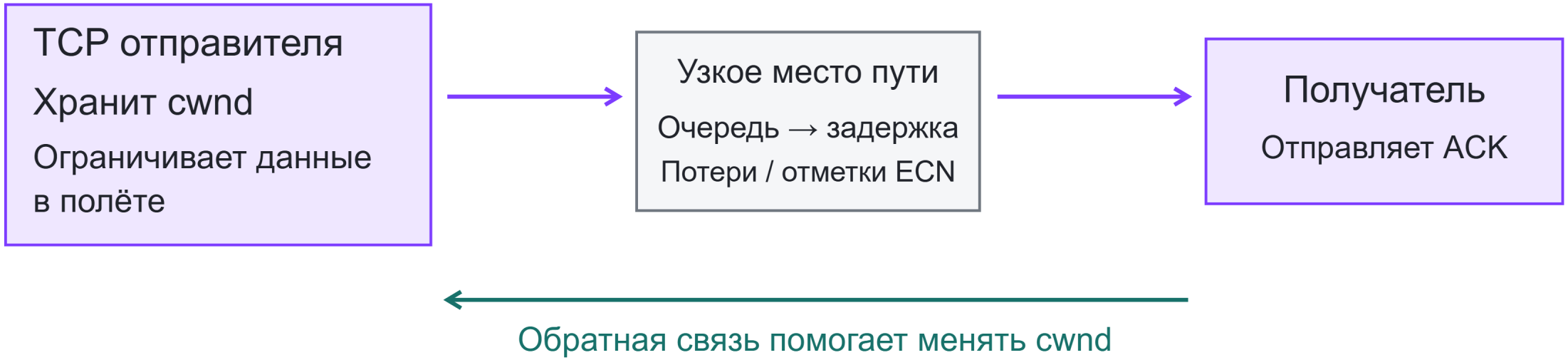
Фиолетовый: непрочитанные данные. Белый: свободное место в учебном буфере.

> TCP может подтвердить приём данных до их чтения приложением. Буфер заполняется — объявляемое окно уменьшается.

При rwnd = 0 отправитель делает контрольные пробы. Получатель объявит ненулевое окно — отправку можно продолжить.

cwnd: сколько отправитель допускает в полёте

Congestion window — окно перегрузки. Даже при свободном буфере получателя путь может не выдерживать нагрузку.



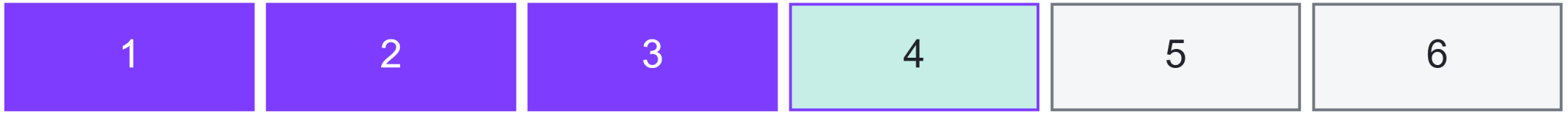
> cwnd хранится у отправителя. Поле Window в заголовке TCP сообщает окно получателя.

ACK и оценки доставки дают обратную связь. Потери и ECN могут вызвать снижение нагрузки; одна потеря ещё не доказывает перегрузку.

Сколько новых данных можно отправить?

Учебная модель: без потерь, окна постоянны, данные готовы к отправке. Один блок = 1000 байт.

rwnd = 6 блоков · cwnd = 4 блока · уже в полёте 3



3 блока ждут ACK

1 можно послать

2 пока сверх cwnd

Пришёл ACK на 2 блока: в полёте осталось 1 → можно послать ещё 3.

> Предел в полёте = min(rwnd, cwnd). Здесь: min(6, 4) – 3 = 1 новый блок.

Если rwnd = 2, cwnd = 8 и в полёте 1 блок, можно добавить только 1: теперь ограничивает получатель. Окно разрешает объём, а не мгновенную отправку.

Как TCP выбирает окно перегрузки

Алгоритм у отправителя определяет, как менять cwnd. Reno — классический алгоритм, на котором удобно объяснять рост окна и реакцию на потери.

Алгоритм	Основная идея
Reno	После начального разгона растит окно примерно линейно за RTT; при обнаружении потерь уменьшает его
CUBIC	Растит окно по кубической функции времени; рассчитан в том числе на быстрые пути с большим RTT
BBR	Оценивает скорость доставки и минимальный RTT; по модели пути регулирует темп отправки и объём данных в полёте

> В модели Reno: slow start — примерно экспоненциальный рост за RTT; после порога ssthresh — линейный рост (congestion avoidance).

ssthresh — порог медленного старта. Окно не удваивается на каждый ACK. Конкретное поведение зависит от алгоритма и его версии.

Широкий канал и объём данных в полёте

Алгоритм меняет cwnd. А какой объём данных нужен для заполнения пути? BDP (bandwidth-delay product) — пропускная способность × RTT.

$$100\ \text{Мбит/с} \times 0{,}1\ \text{с} = 10\ \text{Мбит} = 1{,}25\ \text{МБ}$$

Для заполнения этого пути нужен сопоставимый объём данных в полёте. Если cwnd или rwnd меньше, они могут ограничивать скорость.

Если ждать слишком часто	Если увеличить буферы без разбора
Скорость ограничивает ожидание подтверждений	Можно увеличить очереди и задержку

BDP — характеристика учебного пути, а не алгоритм управления перегрузкой. Оценка без потерь и накладных расходов; это не готовая настройка буфера.

Одна жалоба «медленно», разные наблюдения

Карточка	Наблюдение	Где нужна следующая проверка
A	Окно получателя падает до нуля	Чтение и состояние приложения-получателя
B	RTT высок, данных в полёте мало	Ограничения отправителя и характер обмена
C	Повторы и рост задержки	Обе стороны и состояние пути

> Объём в полёте ограничивают и rwnd, и cwnd. Фактическая отправка зависит также от приложения.

Наблюдения совместимы с несколькими причинами. Следующая проверка должна их различать.

SCTP: сообщения, потоки и несколько адресов

Stream Control Transmission Protocol. Один из примеров применения — сигнализация связи поверх IP.

Свойство	Зачем оно нужно
Границы сообщений	Сохранить единицы прикладного обмена
Несколько потоков	Разделить порядок доставки сообщений
Несколько адресов	Использовать альтернативные пути
Cookie при установлении	Ограничить раннее выделение состояния

> Общие ресурсы остаются. Свойства SCTP сами по себе не гарантируют SLA всей системы.

DCCP: полезная идея, ограниченное распространение

Datagram Congestion Control Protocol: датаграммы с управлением перегрузкой, без повторной передачи потерянных данных.

Препятствие	Почему мешало внедрению
Поддержка на конечных узлах	Приложению и сетевому стеку нужна реализация DCCP
NAT и межсетевые экраны	Многие установленные устройства знали TCP/UDP, но не умели обрабатывать DCCP
Прохождение всего пути	Поддержки на двух концах недостаточно, если промежуточное устройство блокирует обмен

> DCCP не стал массовым транспортом: внедрению мешала совместимость с существующей инфраструктурой.

Для обхода NAT стандартизовали DCCP поверх UDP — RFC 6773. Это снимает часть барьеров, но требует поддержки на концах.

Транспортные свойства можно сочетать иначе

Сравним четыре протокола по свойствам, которые они предоставляют приложению.

Протокол	Единица доставки	Восстановление потерь	Congestion control
TCP	Поток байтов	Да*	Да
UDP	Датаграммы	Нет встроенного	Нет встроенного
SCTP	Сообщения	Да*	Да
DCCP	Датаграммы	Нет для данных	Да

> Границы сообщений, надёжность и управление перегрузкой — отдельные свойства.

* Базовая модель, пока обмен возможен. Расширения и ограничения развёртывания — в лонгриде.

От транспорта к сеансовым задачам

Мы сравнили UDP, TCP, SCTP и DCCP: доставку данных, восстановление потерь и управление нагрузкой.

Теперь поднимемся от транспорта (L4) к организации диалога между приложениями — сеансовым задачам (L5).

> Как вызвать операцию на другом узле и узнать её результат?

Начнём с RPC — удалённого вызова процедуры. Затем рассмотрим организацию телефонного сеанса.

Реальные протоколы могут выполнять задачи нескольких уровней OSI.

Удалённый вызов поверх транспорта

RPC — Remote Procedure Call. Код вызывает процедуру, но между участниками остаётся сеть.

```
Клиент → stub / сериализация → транспорт
       → серверная процедура → ответ
```

Сервер подтвердил байты запроса, затем связь оборвалась. Успел ли он выполнить операцию?

> Отсутствие ответа не различает «не начал», «выполняет» и «выполнил, но ответ потерян».

Stub — локальный представитель удалённого интерфейса. Подробный разбор RPC есть в справочнике.

RPC: карта семейств для дальнейшего чтения

Семейство	Главный ориентир
ONC/Sun RPC, NFS	Процедуры и XDR, файловые операции
Java RMI	Удалённые объекты и методы Java
XML-RPC / SOAP	Вызов в XML / расширяемая модель сообщений
JSON-RPC	Метод, параметры, ID и результат или ошибка
gRPC	Описанный сервис, сообщения и потоки

> Сериализация, транспорт и результат операции относятся к разным частям системы.

XDR — External Data Representation. RMI — Remote Method Invocation. Детали и примеры обменов в лонгриде.

Установить звонок и передать звук

Участники сначала договариваются о сеансе и параметрах, затем обмениваются медиа.

Механизм	Роль
SIP — Session Initiation Protocol	Установление, изменение и завершение сеанса
SDP — Session Description Protocol	Описание согласуемых параметров
RTP — Real-time Transport Protocol	Медиа с номерами и временными отметками
RTCP — RTP Control Protocol	Контрольная информация о медиаприёме

> Сигнализация и медиа могут идти разными путями. Установленный звонок ещё не гарантирует звук.

Где находится логика сеанса?

Пример	Что сравниваем
H.323	Семейство стандартов, бинарная сигнализация и инфраструктурные роли
SIP	Читаемый сигнальный обмен и разные способы распределить функции
IMS и SBC	Примеры значимой роли инфраструктуры в SIP-системах

> «Умный клиент или умная сеть» — вопрос архитектуры. Название протокола не даёт универсального ответа.

IMS — IP Multimedia Subsystem. SBC — Session Border Controller. Подробности H.323 и сравнение с SIP — в справочнике.

Доставка, защита содержимого и доступность

Механизм / риск	Граница
TCP	Не шифрует данные и не удостоверяет прикладную идентичность
TLS	Защищает свой обмен, но не гарантирует доступность транспорта
SYN flood	Давление на обработку и состояние установления
SYN cookies	Пример защиты с ограниченной областью действия

> Поддельные сегменты и перегрузка требуют своей модели защиты. «У нас TLS» не закрывает все транспортные риски.

Вопросы к QUIC на следующей лекции

Что разобрали сегодня	Какой вопрос остаётся
Пропуск в общем TCP-потоке	Можно ли выдавать данные потоков независимо?
TCP, затем TLS	Как иначе организовать защищённое установление?
Соединение с адресами и портами	Как пережить изменение сетевого пути?

> QUIC реализует транспортные функции поверх UDP. UDP не добавляет их автоматически.

Разберём также общие ограничения потоков, 0-RTT, HTTP/3 и fallback.

Проверка понимания

1. TCP подтвердил запрос, но ответа приложения нет. Что мы знаем о выполнении операции?
2. Получатель объявил окно ноль. Где искать следующее наблюдение?

На семинаре	В лонгриде
Сборка сообщений, Seq/Ack, разбор «медленно» и диагностический отчёт	Механизмы TCP, справочник RPC и протоколов связи, дополнительные примеры

> Полезный ответ отделяет наблюдение от предположения и предлагает различающую проверку.